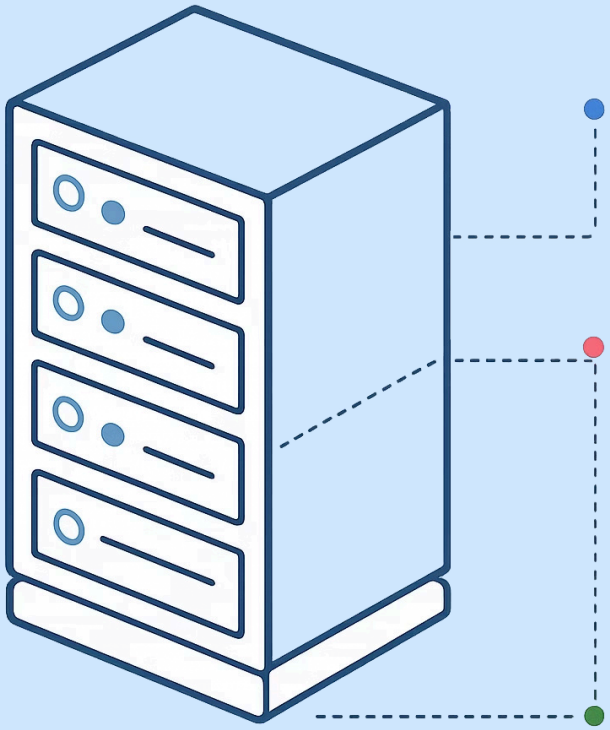
Multi Container Runtime

An OS mini project exploring Linux Namespaces, Pipes & IPC, Kernel Modules, CFS Scheduling, and POSIX Synchronization.

DHRUV JAGADEESH HEMMIGE · PES2UG24AM053

NIRAJ KUMAR · PES2UG24AM046



System Architecture

All four systems interlock: namespaces isolate, pipes stream, buffer absorbs, kernel enforces.

engine.c

User-Space Supervisor

- Container lifecycle & CLI
- Logging pipeline
- Signal handling
- select() event loop

monitor.ko

Kernel Module

- RSS enforcement
- Timer callback / softirq
- ioctl interface
- Spinlock protection

Bounded Buffer

Producer-Consumer Queue

- Decouples pipe reads from disk I/O
- mutex + cond var
- Backpressure via blocking producers

Namespaces

PID + UTS + Mount

- Per-container process tree
- Private hostname
- Isolated mount table

Linux Namespaces & Isolation

A namespace is a kernel abstraction that makes a process believe it has an exclusive, private view of a system resource. This runtime uses three: **PID, UTS, and Mount** — sufficient to demonstrate meaningful isolation.

Namespace	Clone Flag	What It Isolates
PID	`CLONE_NEWPID`	Process IDs — container starts at PID 1
UTS	`CLONE_NEWUTS`	Hostname and NIS domain name
Mount	`CLONE_NEWNS`	Filesystem mount table; chroot becomes safe

clone() — Creating the Container Process

PID Mapping Example

Host View: PID 1 = systemd · PID 4521 = engine · PID 4522 = sh (alpha)

Container View: PID 1 = /bin/sh ← same kernel PID · PID 2 = cpu_hog · (no visibility above PID 1)

```
pid_t pid = clone(
 child_fn, // container entry point
 stack_top, // x86 stacks grow DOWN
 CLONE_NEWPID |
 CLONE_NEWUTS |
 CLONE_NEWNS |
 SIGCHLD, // enables waitpid()
 &cfg
);
```

Pipes & Inter-Process Communication

Channel A — Pipe (log data)

container stdout/stderr → 64 KiB FIFO pipe buffer → supervisor reader thread

`pipe2(pipefd, O_CLOEXEC)` · `dup2(pipefd[1], STDOUT_FILENO)`

⚠ **Critical:** `close(pipefd[1])` in parent — EOF occurs ONLY when ALL write ends are closed. If parent holds it open, reader blocks forever.

Channel B — UNIX Socket (CLI control)

`/tmp/mini_runtime.sock` — handles start / stop / ps / logs

Fixed-size packed structs with `__attribute__((packed))` prevent padding mismatches. Kernel passes peer PID/UID via `SO_PEERCRED`.

Bounded Buffer — Producer-Consumer Sync

Mutex + Condition Variable

Mutex: Binary lock — only one thread modifies head/tail/count at a time.

Condition Variable: `pthread_cond_wait()` atomically releases mutex AND sleeps — eliminates the lost wakeup bug.

while() not if(): POSIX permits spurious wakeups. `while()` rechecks the predicate every time, preventing overflow on false alarms.

```
bb_push: lock → while(count==CAP)
 cond_wait(¬_full, &mutex)
 → insert → count++
 → signal(¬_empty) → unlock
```

Kernel Module: monitor.ko

User Space (ring 3): engine.c — protected; can crash safely. **Kernel Space (ring 0):** monitor.c — one bad pointer = kernel panic.

CPU Starvation Immunity

Timer fires regardless of user-space scheduling.

Compromise Resistance

Container cannot ptrace() or kill a kernel timer.

Authoritative Data

Reads directly from mm_struct — kernel ground truth.

Unstoppable SIGKILL

force=1 bypasses signal blocking; delivered next timeslice.

Limit	Threshold	Action
Soft	soft_limit_bytes	Log kernel warning (once per container)
Hard	hard_limit_bytes	send_sig(SIGKILL, task, 1) — force=1 bypasses blocking

Signal Handling & Async-Signal-Safety

Signal	Used For
SIGCHLD	Reap dead containers via waitpid()
SIGTERM/SIGINT	Graceful shutdown — drain logs, stop containers
SIGPIPE	Ignored (SIG_IGN) — write() returns EPIPE
SIGKILL	Sent by kernel monitor on hard limit breach

⚠ NEVER call malloc/free, printf, or FILE* I/O inside a handler. Use the flag pattern: handler sets `volatile sig_atomic_t`; real work runs in the event loop.

Flag Pattern

```
volatile sig_atomic_t g_got_sigchld = 0;

static void sigchld_handler(int sig) {
    g_got_sigchld = 1; // ONLY safe op
}

// In event loop:
if (g_got_sigchld) {
    g_got_sigchld = 0;
    reap_children();
}
```

waitpid loop required: Linux may coalesce multiple SIGCHLD signals. Without the loop, unreaped zombies accumulate until the PID table exhausts.

CFS Scheduling & Supervisor Event Loop

CFS assigns CPU proportional to weight derived from nice value. Task with smallest vruntime runs next. Use `setpriority()` not `nice()` — the latter returns -1 on both success and error.

Event Loop Architecture

Main Thread

select() on control socket (1s timeout). Accepts CLI, checks SIGCHLD flag, handles shutdown.

Logger Thread

bb_pop() from bounded buffer → write() to .log. Drains all items on shutdown.

Reader Thread

One per container. read() from pipe → bb_push(). Exits on EOF.

select() vs epoll(): ⩽33 fds — O(n) cost is negligible. epoll adds ~150 LOC for zero measurable gain at this scale.

Design Tradeoffs & Key Lines

Design Decisions

PID+UTS+Mount only

Network ns needs veth/bridge; User ns needs UID mapping. Three is enough to teach isolation.

Spinlock in kernel

Timer runs in softirq — cannot sleep. mutex_lock() in softirq causes kernel panic.

mutex+cond var buffer

Lock-free rings need arch-specific barriers. Correctness matters more than µs latency.

Packed fixed structs

Zero serialization overhead, no external library, padding-free across compilers.

Lines You Must Explain

- → `clone(...SIGCHLD...)` → enables waitpid()
- → `close(pipefd[1])` in parent → EOF detection
- → `while() cond_wait` → spurious wakeup safety
- → `copy_from_user()` → different page tables
- → `spin_lock` not mutex in softirq → cannot sleep
- → `chroot() + chdir('/')` → seal escape path